ECE551: Digital System Design and Synthesis

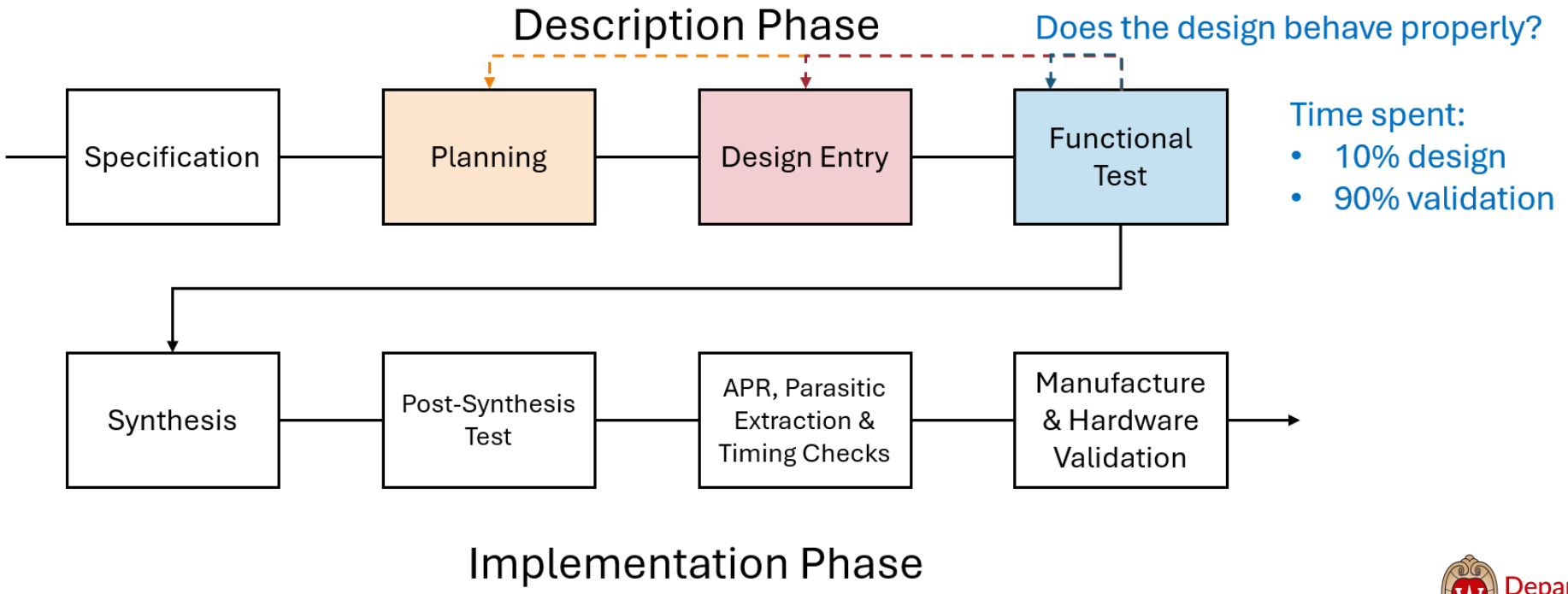
Fall 2026

Testbench Constructs

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Testbench Constructs

- We spend 90% of our design time on testing & debugging
- Important to have good & efficient testbenches!
  - Modular
  - Reusable
  - Less code to do more



fork/join

- We use **begin/end** pair to form compound sequential statements in procedural
- **fork/join** are used to form compound parallel statements
  - Statements in a parallel block are executed simultaneously
  - All delay or event-based control is relative to when the block was entered

```
begin
  #5  r = 'h35;
  #10 r = 'hE2;
  #15 r = 'h00;
  #20 r = 'hF7;
end
```

vs

```
fork
  #5  r = 'h35;
  #10 r = 'hE2;
  #15 r = 'h00;
  #20 r = 'hF7;
join
```

vs

```
fork
  #20 r = 'hF7;
  #15 r = 'h00;
  #10 r = 'hE2;
  #5  r = 'h35;
join
```

- Can be useful when you want to wait for the occurrence of a few events, but you don't know the order the events will occur

```
begin
  fork
    @Aevent
    @Bevent
  join
  areg = breg;
end
```

- Could be a source of races…

```
fork
  #5 a = b;
  #5 b = a;
join
```

→

```
fork
  a = #5 b;
  b = #5 a;
join
```

Named Blocks

- Blocks (begin/end or fork/join) can be named
  - Local variables can be declared for the named block
  - Variables in a named block can be accessed using hierarchical naming reference
  - Named blocks can be disabled (i.e., execution stopped)

```
module top();

initial begin: block1
  integer i;     // i is local to block1
  ...

end              // end of block1

endmodule
```

block name

Can be referenced using **top.block1.i**

```
always_ff @(posedge clock) begin: Sequencer
  case (SquatState)
    2'b01: begin: rx_valid_state
             Rxready <= '1;
             for (int j=0; j<NumRx; j++) begin: loop1
               for (int i=0; i<NumRx; i++) begin: loop2
                 if (Rxvalid[i]) begin: match
                   ... // receive data
                 end: match
               end: loop2
             end: loop1
           end: rx_valid_state
    ... // decode other states
  endcase
end: Sequencer
```

Example taken from Sutherland/Mills Synthesizing System Verilog paper (posted and worth reading)

Named Blocks

- Blocks (begin/end or fork/join) can be named
  - Local variables can be declared for the named block
  - Variables in a named block can be accessed using hierarchical naming reference
  - Named blocks can be disabled (i.e., execution stopped)
    - **disable** statements provide execution control similar to **break** and **continue** in C

```
begin : break
  for (i = 0; i < n; i = i+1) begin : continue
    @(posedge clk)
    if (a == 0) // "continue" loop
      disable continue;
    if (a == b) // "break" from loop
      disable break;
    statement1
    statement2
  end : continue
end : break
```

Handy Use of **fork/join** and **disable** of a Named Block

- A UART transmitter is sending a command to a UART receiver. ***cmd_rdy*** will go high when the reception is complete.

```
@(posedge cmd_rdy)
```

What if ***cmd_rdy*** never goes high?
Our test bench will hang!

```
@(negedge clk);
rst_n = 1;

@(negedge clk);
send_cmd = 1;
@(negedge clk);
send_cmd = 0;
```

Using **fork/join** and **disable** we can make a test bench that will wait for ***cmd_rdy***, but also time out if it never occurs.

```
fork
  begin : timeout1          // This block will error out after 70k clocks
    repeat(70000) @(posedge clk);
    $display("ERROR: timed out waiting for transmission to complete");
    $stop();
  end
  begin                     // This block waits for cmd_rdy
    @(posedge cmd_rdy);
    disable timeout1;       // Cancels timeout as soon as cmd_rdy occurs
  end
join
```

Assertions

- Self-checking testbenches are a must

```
if (result === expected)  $display(“self check passed”)
else begin
  $display(“ERR: at time %t, result not same as expected”,$time);
  $finish();
end
```

- System Verilog offers an assert statement to help simplify this self check
- General syntax:

```
assert (true_condition) pass_statement
else fail_statement
```

```
assert (result === expected)  $display("self check passed");
else $fatal(1,"ERR: at time %t, result not same as expected",$time);
```

pass_statement

fail_statement

Assertions

- Self-checking testbenches are a must

```
if (result === expected)  $display("self check passed");
else begin
  $display("ERR: at time %t, result not same as expected",$time);
  $finish();
end
```

- System Verilog offers an assert statement to help simplify this self check
- General syntax:

```
assert (true_condition) pass_statement
else fail_statement
```

```
assert (result === expected)  $display("self check passed");
else $fatal(1,"ERR: at time %t, result not same as expected",$time);
```

fail_statement can be compound using **begin/end**

```
else begin
    $error("ERR: at time %t, result not same as expected",$time);
    $stop();
end
```

Assertions

- **Immediate assertions** are evaluated when the statement is reached in the testbench flow. They're just a more succinct **if** + **$display** self-check.

```
assert (result === expected)  $display("self check passed");
else $fatal(1,"ERR: at time %t, result not same as expected",$time);
```

- **Concurrent assertions** allows you to define conditions that should **always** be true and are checked at all times during simulation.

```
/// check that rd & wrt are never both asserted   ///
/// This will be checked at every simulation tick ///
assert property (!(rd && wrt))
else $error("ERROR: both rd and wrt asserted to SRAM");
```

The keyword property distinguishes a concurrent assertion from an immediate assertion.

Concurrent Assertions

- Most concurrent assertions would be checked on clock ticks.

```
/// when req is asserted, ack should be asserted 1 to 2 clocks later ///
assert property (@(posedge clk) req |-> ##[1:2] ack);
```

specifies the clock associated with this concurrent property

evaluated just prior to clock rise

Concurrent Assertions

- Most concurrent assertions would be checked on clock ticks.

```
/// when req is asserted, ack should be asserted 1 to 2 clocks later ///
assert property (@(posedge clk) req |-> ##[1:2] ack);
```

antecedent sequence

consequent sequence

- Assertion has a few operators you can use to build more complex checks
  - The **implication** operator (*antecedent* `|->` *consequent)*: whenever the *antecedent sequence* occurs, the *consequent sequence* must become true.

Concurrent Assertions

- Most concurrent assertions would be checked on clock ticks.

```
/// when req is asserted, ack should be asserted 1 to 2 clocks later ///
assert property (@(posedge clk) req |-> ##[1:2] ack);
```

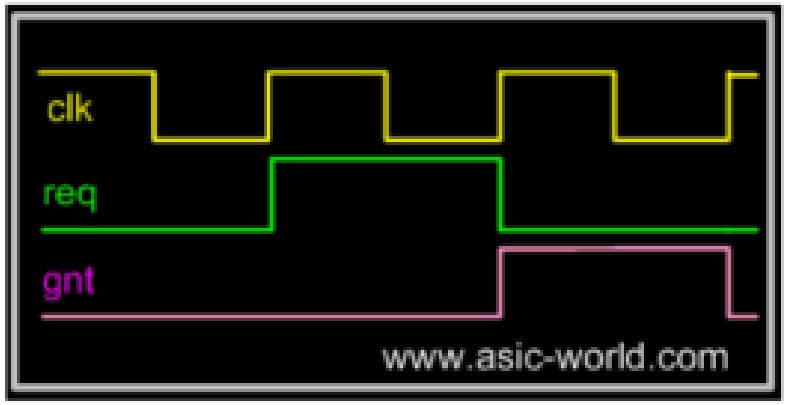
ranged delay: 1 to 2 clock ticks after current clock

- Assertion has a few operators you can use to build more complex checks
  - The **implication** operator (*antecedent* `|->` *consequent)*: whenever the *antecedent sequence* occurs, the *consequent sequence* must become true.
  - The **delay** operator (##): the following sequence starts a set number of clock cycles after the current clock tick
    - ##1: one clock later
    - ##[1:2]: one to two clocks later (a range)
    - ##[0:3]: zero to three clocks later (0 means the same cycle)

```
req |-> ##[0:3] gnt
```

Concurrent Assertions

- Assertions can get rather complex, don't have to specify all in one line
  - Can break assertions into multiple parts: sequences, properties, and directive




```
////Sequence Layer////
sequence req_gnt_seq;                  // clock right after req, req
  (~req & gnt) ##1 (~req & ~gnt);      // should be low and gnt should
endsequence                            // be high. Next clock both low

////Property Layer////
property req_gnt_prop;
  @(posedge clk)                       // clk is used for clock ticks
    disable iff (!rst_n)               // will not check when resetting
    req |-> ##1 req_gnt_seq;           // upon req the sequence req_gnt_seq
endproperty                            // specified above should occur

////Directive Layer////
assert property (req_gnt_prop)
else $display("ERR: req_gnt assertion failure at time %t",$time);
```

Loops

- Syntax supports different loop “types”
  - for
  - repeat
  - forever
  - while
  - do … while
  - foreach

for Loops

```
reg [15:0] rf[0:15];  // memory structure for modeling register file
reg [4:0] w_addr;     // address to write to

initial
  for (w_addr=0; w_addr<16; w_addr=w_addr+1)
     rf[w_addr[3:0]] = 16'h0000; // initialize register file memory
```

```
integer i;
for (i = 0; i < 8; i++) begin
    ...
end
```

- Looks the most C-like → easier to read
- You get to specify an initial value, terminating condition, and how it is updated all in one line
- But you need to manage the iterator explicitly, so may be error prone

  → recommendation: use integer if it's a plain iterator

When to use?

- Good for counted iterations
- Good for **generate for**

```
genvar i;

generate
  for (i = 0; i < N; i++) begin : stage
    full_adder iFA(a[i], b[i], cin[i], sum[i], cin[i+1]);
  end
endgenerate
```

Aside: Generated Instantiations

- Generate statements control the instantiation / creation of
  - Modules
  - Gate primitives
  - Continuous assignments
  - Initial blocks and always blocks
- Resolved during "elaboration" (compile time)
  - When module instantiations are linked to module definitions
  - Before the design is simulated or synthesized → this is NOT dynamically created hardware

```
module gray2bin1 (bin, gray);
  parameter SIZE = 8;
  output [SIZE-1:0] bin; input [SIZE-1:0] gray;

  genvar i;
  generate
     for (i=0; i<SIZE; i=i+1) begin: bit
        assign bin[i] = ^gray[SIZE-1:i];
     end
  endgenerate
endmodule
```

Does not exist during simulation of design

Typically name the generate as reference

...bit[0].bin[0]

```
assign bin[0] = ^gray[7:0]; // bit[0]
assign bin[1] = ^gray[7:1]; // bit[1]
assign bin[2] = ^gray[7:2]; // bit[2]
...
assign bin[7] = ^gray[7:7]; // bit[7]
```

Aside: Generated Instantiations

- Generate-conditional
  - Allows conditional instantiation using **if…else if…else**

```
module multiplier(a ,b ,product);
  parameter A_WIDTH = 8, B_WIDTH = 8;
  localparam PRODUCT_WIDTH = A_WIDTH + B_WIDTH;
  input [A_WIDTH -1:0] a; input [B_WIDTH -1:0] b;
  output [PRODUCT_WIDTH-1:0] product;

  generate
    if ((A_WIDTH < 8) || (B_WIDTH < 8))
      CLA_multiplier #(A_WIDTH, B_WIDTH) u1(a, b, product);
    else
      WALLACE_multiplier #(A_WIDTH, B_WIDTH) u1(a, b, product);
  endgenerate
endmodule
```

These are **parameters**, not variables!

- Generate-case
  - Allows conditional instantiation using **case**

```
module adder (output co, sum, input a, b, ci);
   parameter WIDTH = 8;
   generate
      case (WIDTH)
        1: adder_1bit x1(co, sum, a, b, ci); // 1-bit adder impl.
        2: adder_2bit x1(co, sum, a, b, ci); // 2-bit adder impl.
        default: adder_cla #(WIDTH) x1(co, sum, a, b, ci);
      endcase
   endgenerate
endmodule
```

Aside: Generated Instantiations

- Sometimes we need a way to accelerate the function of a block to make it practical to simulate.
- Use a parameter to control whether a block uses its full logic or a faster, abbreviated version.

```
module IR_intf(input clk, rst_n, line_present, output [11:0] IR_R0, …);

  parameter FAST_SIM = 0;

  ...

  generate if (FAST_SIM) begin
    assign next_round = &timer[13:0];  // smaller 14-bit timer
    assign settled = &timer[10:0];
  end else begin
    assign next_round = &timer;        // the full 17-bit timer
    assign settled = &timer[12:0];
  endgenerate

endmodule
```

repeat Loops

```
initial begin
  inc_DAC = 1'b1;
  repeat(4095) @(posedge clk); // bring DAC right up to point of rollover
  inc_DAC = 1'b0;
  inc_smpl = 1'b1;
  repeat(7) @(posedge clk);    // bring sample count up to 7
  inc_smpl = 1'b0;
end
```

```
repeat(3) begin
  ...
end
```

- Good for fixed number of iterations
  - Repeat count can be a variable, but it is only evaluated when the loop starts
    → if its value changes during loop execution, it won't change the number of iterations
- Handy & succinct way to wait for a fixed number of clocks when used with @(posedge clk)

forever Loops

```
initial begin
  clk = 0;
  forever #10 clk = ~ clk;
end
```

- Only a **$stop**, **$finish** or a specific **disable** can end a forever loop
- Clock generator is the most common use of a forever loop

while / do...while Loops

- Executes while the condition is true
  - while → If condition is false from the beginning, it will never execute the loop
  - do … while → always executes 1 iteration first, then check for condition

```
reg [15:0] flag; reg [4:0] index;

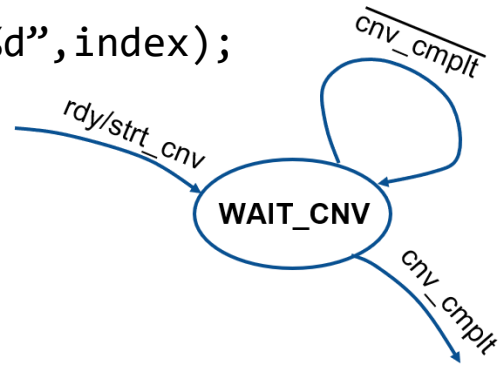
initial begin
  index=0; found=1'b0;
  while ((index<16) && (!found)) begin
    if (flag[index]) found = 1'b1;
    else index = index + 1;
  end
  if (!found)
    $display("non-zero flag bit not found!");
  else
    $display("non-zero flag bit found in position %d",index);
end
```

Handy for cases where the terminating condition is more complex than i<N

* Need to make sure condition **will** become false, otherwise infinite loop!

- Good for polling logic in testbench

  → avoid in DUT code (should think FSM instead)



foreach Loops

- Iterates through every entry of an array

```
module sqrt_table(clk,addr,dout,wdata,we);
  parameter DEPTH = 65536;  // default to 16-bit numbers
  ...
  logic [$clog2(DEPTH)/2-1:0] sqrt_entry[0:DEPTH-1];

  initial
    foreach (sqrt_entry[i])
      sqrt_entry[i] = $rtoi($sqrt(i));

  ...
endmodule
```

i is automatically declared and managed

Code is independent of DEPTH

- Used only for array traversal

foreach Loops

- Iterates through every entry of an array

```
module sqrt_table(clk,addr,dout,wdata,we);
  parameter DEPTH = 65536;  // default to 16-bit numbers
  ...
  logic [$clog2(DEPTH)/2-1:0] sqrt_entry[0:DEPTH-1];

  initial
    foreach (sqrt_entry[i])
      sqrt_entry[i] = $rtoi($sqrt(i));

  ...
endmodule
```

- Used only for array traversal

- Works with multi-dimensional arrays

```
module mult_table (
    input  logic [3:0] a, b,
    output logic [7:0] product
);
    logic [7:0] mult_lut [0:15][0:15];

    initial begin
        foreach (mult_lut[i][j])
            mult_lut[i][j] = i * j;
    end

    assign product = mult_lut[a][b];
endmodule
```